

10-Day Campus Interview Prep Plan

High-impact DSA topics: Campus interview questions at major product companies cover a wide range of algorithms and data structures. For example, GeeksforGeeks notes that interviews “cover a wide range of topics including ... arrays, strings, dynamic programming, and more” [3†L28-L36] . Similarly, candidates report seeing a *mix* of arrays, strings, linked lists and trees, with graphs and dynamic programming (“DP”) often tested for senior roles [12†L218-L224] . Given your background (sliding window, binary search, bit tricks, graphs, DP are solid), focus the next 10 days on any remaining Striver topics: e.g. **arrays/strings, stacks, queues, linked lists, recursion/backtracking, trees/tries, greedy algorithms, and hashing**. Prioritize two-pointer/sliding-window and stack/queue problems first (as these are common “baseline” topics) [12†L150-L159] . Then tackle greedy/DP and tree/graph problems (which are higher difficulty). Practice a few problems daily on each topic (e.g. 4–5 easy/medium problems on two-pointers, stacks, etc.) and review the core concepts (hash-maps for anagrams, recursion for backtracking, etc.). This ensures coverage of all Striver sheet areas.

- **Arrays/Strings:** Two-pointer, sliding-window, hashing (anagrams, subarrays); common pattern problems [12†L218-L224] .
- **Linked Lists:** Reverse, cycle detection, merge, rearrange, random-pointer; often asked by Amazon/Microsoft.
- **Stacks/Queues:** Monotonic stack patterns, queue (rotating, sliding max); e.g. “Next Greater Element”, “First non-repeating in stream”.
- **Trees/Graphs:** Tree traversals, LCA, BFS/DFS on grids, shortest paths; critical for many companies.
- **Dynamic Programming/Greedy:** Knapsack, LIS/LCS, DP on strings; greedy interval or scheduling. (You’ve done DP already, so do one or two complex problems to reinforce.)
- **Recursion/Backtracking:** Subset generation, permutations, sudoku, N-queens; good for showing thought process.
- **Bit Manipulation/Math:** You’re strong here; do a couple of practice problems to stay sharp.
- **Hashing/Tries:** String hashing, prefix trees (e.g. autocomplete); common interview questions on strings and prefixes.
- **Operating Systems:** Briefly review *key OS concepts* (process vs thread, scheduling, memory management, concurrency) since campus interviews often cover fundamentals [19†L84-L86] .

Low-Level Design (LLD) practice: In parallel, spend time on design. Focus on a handful of common LLD problems and design patterns rather than trying to cover everything at once. For instance, interview prep experts recommend practicing ~5–10 classic design scenarios such as a parking lot, elevator, tic-tac-toe, snake-and-ladder game, LRU cache, ride-sharing service, or hotel booking system [14†L82-L90] . These span simple OO design (games, cache) to small system components. Drill down one or two designs per day: sketch class diagrams, explain SOLID principles, use design patterns (e.g. Singleton, Observer, Factory) as appropriate. Leverage resources like the [GitHub LLD problems list] or tutorials (e.g. Hello Interview, Gaurav Sen) and **don’t just watch solutions** – code or draw them yourself first [14†L82-L90] .

First 5 Days: React + Daily Contests

During the 5-day React learning period (2–3 hours/day), continue coding practice by doing **one virtual contest each day** (e.g. CodeChef, Codeforces, or any mock contest platform). This maintains momentum and simulates pressure. Try to solve 2–3 problems per contest (easy/medium level) and review any mistakes afterwards. Daily contests help identify weak spots and build speed for interviews. In total you'll spend ~2–3 hrs on React and ~1–2 hrs on contest practice each day.

Remaining 10 Days: Structured Schedule

For the final 10 days, alternate between DSA problem practice, LLD design work, mock contests, and brief OS review. A sample breakdown might be:

1. **Day 6 (DSA Deep Dive):** Focus on **Stacks & Queues**. Solve 4–5 problems (e.g. “Next greater element”, “Sliding window maximum”, “LRU Cache” coding). Review any related OS concepts (e.g. multithreading vs multiprocessing) in 30 mins 【19†L84-L86】 .
2. **Day 7 (DSA Linked Lists):** Solve 3–4 linked-list problems (reverse list, detect/remove loop, merge k lists). Do one contest-style problem from your weak topic.
3. **Day 8 (Contest + Trees):** Take a short timed mock contest (2–3 problems from Codeforces/LeetCode) to simulate interviews. Then study **Trees/Graphs**: implement traversals and solve problems like lowest common ancestor or bipartite graph check.
4. **Day 9 (DSA Tries & Greedy):** Work on **Tries/Hashing** (e.g. word break, auto-complete) and revisit **Greedy** (interval scheduling, knapsack).
5. **Day 10 (LLD Practice #1):** Pick one system design to code/diagram (e.g. **Design LRU Cache** or **Elevator System**). Write out classes/methods; enforce SOLID principles.
6. **Day 11 (Recursion/DP):** Solve a couple of backtracking problems (e.g. permutations, combination sum) and revisit one DP problem (e.g. edit distance or subset sum).
7. **Day 12 (LLD Practice #2):** Second design topic (e.g. **Design Parking Lot** or **Ticket Booking System**). Focus on handling concurrency or threads if relevant (booking systems often require locking threads) 【14†L82-L90】 .
8. **Day 13 (Contest + OS Review):** Another mock contest session. Then quickly review any OS topics you find shaky (CPU scheduling algorithms, deadlocks, memory management) using interview Q's summaries 【19†L84-L86】 .
9. **Day 14 (Mixed Revision):** Identify two weakest areas from earlier (maybe Graphs or DP) and do targeted practice problems.
10. **Day 15 (Final Brush-Up):** Do a short daily quiz or flashcards on key topics (SOLID principles, common data structures). Skim through Striver sheet summaries or quick notes. Practice one more contest problem or explain a design verbally to a peer.

Throughout, **review solutions and rationalize each approach**. As one experienced interviewer advises: focus on *understanding logic* and explaining clearly 【3†L28-L36】 【14†L82-L90】 . In summary, allocate each day to a couple of concrete tasks (e.g. solve 4 DSA problems or design 1 system) and alternate with contests and mini-OS reviews. This balanced plan – daily contest + targeted topic practice + LLD study – will prepare you for campus interviews at product companies 【3†L28-L36】 【14†L82-L90】 .

Sources: Top companies' interviews frequently test the topics above 【3†L28-L36】 【12†L218-L224】 , and experts recommend drilling common design problems and principles 【14†L82-L90】 . Review OS

fundamentals (scheduling, concurrency, memory) as they are often asked 【19†L84-L86】 . Following a structured schedule with daily practice is key to success 【12†L150-L159】 【14†L82-L90】 .
